

## Response B



# PostgreSQL Integration Test Diff Review

## 1. Summary

This PR adds PostgreSQL support to the canonical expectations test suite, enabling verification that core expectations work across database backends. The implementation creates temporary tables, inserts test data, and cleans up afterward. However, there are **critical resource management issues** that will cause connection leaks and test flakiness in CI. The random table naming strategy is insufficient for parallel test execution, and the lack of proper connection/transaction cleanup will eventually exhaust the database connection pool.

## 2. Blockers

### Blocker 1: Connection Leak in `setup()` Method

**Location:** `tests/integration/test_utils/data_source_config/postgresql.py`, lines 176-183

#### What's wrong:

Python

```
1
2
3
```

The connection context manager doesn't guarantee transaction cleanup in all error paths, and more critically, the `self.engine` is never disposed of properly.

#### Why it matters:

PostgreSQL has connection limits. In a CI environment running parallel tests, undisposed engines will accumulate connections until the database refuses new connections, causing random test failures.

Each **Engine** maintains a connection pool that must be explicitly disposed.

#### Fix:

Add `self.engine.dispose()` to the `teardown()` method:

```
Python
```

```
1  
2  
3  
4  
5
```

## Blocker 2: Insufficient Isolation for Parallel Test Execution

**Location:** `tests/integration/test_utils/data_source_config/postgres.py` , line 147

**What's wrong:**

```
Python
```

```
1
```

With only 1 million possibilities and random selection, birthday paradox makes collisions likely in parallel test runs (approximately 1% chance with just 100 concurrent tests).

**Why it matters:**

In CI with parallel pytest execution (e.g., `pytest-xdist` ), table name collisions will cause tests to fail unpredictably when one test tries to create a table that another test is using. This creates flaky tests that are extremely difficult to debug.

**Fix:**

Use a UUID or combine worker ID with timestamp:

```
Python
```

```
1  
2
```

Or better yet, use pytest's worker ID if available:

Python

```
1  
2  
  
3
```

### Blocker 3: Teardown Not Guaranteed on Test Failure

**Location:** `tests/integration/test_utils/data_source_config/base.py`  
(inherited behavior, affects `postgres.py`)

#### What's wrong:

There's no visible fixture or context manager ensuring `teardown()` is called if `setup()` or `make_batch()` raises an exception. The diff doesn't show the `BatchTestSetup` base class implementation fully, but the pattern suggests manual lifecycle management.

#### Why it matters:

If a test fails during execution or setup, the table may never be dropped, leading to database pollution. Over time, this accumulates garbage tables that consume disk space and may interfere with subsequent test runs.

#### Fix:

The `BatchTestSetup` should implement `__enter__` and `__exit__` to ensure cleanup:

Python

```
1  
2  
3  
4  
5  
  
6  
7
```

And usage in tests should be:

Python

```
1  
2  
3
```

## Blocker 4: No Transaction Rollback on Test Failure

**Location:** `tests/integration/test_utils/data_source_config/postgres.py`, line 177

### What's wrong:

The connection commits data immediately without transaction isolation between setup and teardown. If teardown fails, partial data remains.

### Why it matters:

If `conn.commit()` succeeds but `table.drop()` later fails (e.g., due to active connections from the datasource itself), the table persists with data, potentially affecting other tests or consuming resources.

### Fix:

Wrap the entire operation in a transaction and use explicit begin/rollback:

Python

1

2

## Blocker 5: Missing Error Handling for Table Drop

**Location:** `tests/integration/test_utils/data_source_config/postgres.py`, lines 187-189

### What's wrong:

Python

1

2

3

4

No exception handling if the table drop fails (e.g., due to locked connections or permissions).

### Why it matters:

A failing `teardown()` will mask the actual test failure and make debugging harder. Additionally, the table persists in the database, leading to accumulation over multiple test runs.

**Fix:**

Add explicit error handling and logging:

Python

```
1
2
3
4
5
6
7

8
9

10
11
```

### 3. Non-blocking Suggestions

#### Suggestion 1: Add Connection String Validation

**Location:** `tests/integration/test_utils/data_source_config/postgres.py`, line 149

The hardcoded connection

string `postgresql+psycopg2://postgres@localhost:5432/test_ci` should be:

- Read from an environment variable with a fallback
- Validated before attempting connection
- Documented in test setup instructions

Python

```
1
2
3
4

5
```

#### Suggestion 2: Type Annotation Improvements

**Location:** `tests/integration/test_utils/data_source_config/postgres.py`, line 152

Python

1

Should use modern optional syntax:

Python

1

Suggestion 3: Add Docstrings

The `PostgreSQLDataSourceTestConfig` and `PostgresBatchTestSetup` classes lack docstrings explaining their purpose, usage patterns, and the lifecycle contract.

Suggestion 4: Consistent Schema Naming

Location: Line 152

The schema is hardcoded as `"public"` . Consider making it configurable or documenting why it's always public.

Suggestion 5: Import Organization

Location: Lines 1-57

The massive import block for PostgreSQL types could be better organized. Consider:

Python

1

2

Suggestion 6: Data Conversion Robustness

Location: Line 181

Python

1

This assumes all pandas dtypes are compatible with the specified SQL types. Add validation or type coercion to handle edge cases (e.g., pandas nullable integers, timestamps with timezone info).

## 4. Tests/Validation

To ensure robustness, the following test scenarios should be validated:

### Test Scenario 1: Parallel Execution

- Run the test suite with `pytest-xdist -n 4` (4 workers)
- Verify no table name collisions occur
- Check that connection pool doesn't exhaust

### Test Scenario 2: Failure During Setup

- Mock `metadata.create_all()` to raise an exception
- Verify teardown still runs and cleans up the engine
- Confirm no orphaned connections remain

### Test Scenario 3: Failure During Teardown

- Mock `table.drop()` to raise an exception
- Verify the test failure is reported correctly
- Ensure engine is still disposed

### Test Scenario 4: Connection Pool Exhaustion

- Run 100+ tests sequentially
- Monitor PostgreSQL connection count ( `SELECT count(*) FROM pg_stat_activity` )
- Verify connections return to baseline after tests complete

### Test Scenario 5: Type Compatibility

- Test with all commented-out SQLAlchemy 2.x types when upgrading
- Verify complex types (ARRAY, JSONB, HSTORE) serialize correctly from pandas
- Test NULL handling for all types

### Test Scenario 6: Database Unavailable

- Stop PostgreSQL before running tests

- Verify graceful failure with clear error message
- Confirm tests are properly skipped with `pytest.skip` if DB unavailable

## Test Scenario 7: Cleanup Verification

- After test run, query `information_schema.tables` for leftover tables
- Verify no tables matching the test pattern remain
- Check for orphaned schemas

## Test Scenario 8: Large Data Sets

- Test with DataFrames containing 10K+ rows
- Verify memory efficiency of `to_dict("index")`
- Confirm no connection timeout during insert

## Test Scenario 9: Special Characters in Data

- Test data containing SQL injection patterns, unicode, null bytes
- Verify proper escaping and no SQL errors
- Confirm data round-trips correctly

## Test Scenario 10: Concurrent Access

- Have one test read from a table while another tries to drop it
- Verify proper locking behavior
- Ensure no deadlocks occur

---

**Recommendation:** This PR should **not be merged** until Blockers 1, 2, 3, and 5 are resolved. The connection leak and collision issues will cause immediate CI instability. After fixes, extensive parallel testing should be performed before merging.